MANUEL DE L'ÉQUIPE CHAIN619

EL JOKER, expliqué depuis zéro

EL JOKER, c'est **la couche de contrôle pour les agents IA qui dépensent de l'argent** : 5 agents logiciels qui achètent pour une organisation (une ONG en urgence, une entreprise qui équipe ses équipes). Un seul agent utilise l'IA, pour conseiller. Les quatre autres sont du code strict qui vérifie, décide et paie. Au-dessus de la limite, l'argent est bloqué dans un **coffre Hedera** que seule une signature humaine peut ouvrir. Chaque décision laisse une preuve sur **Hedera**, un registre public que personne ne peut modifier, même pas vous.

L'IA propose | **Les règles décident** | **Hedera prouve**

Commence par le chapitre 2 : c'est le simulateur qui rejoue exactement ce que fait le site quand tu envoies une demande.

CHAPITRE 1

L'idée, en vraie vie

Imagine une ONG à Gabès. Le réseau d'eau tombe en panne, 500 personnes ont besoin d'eau et de médicaments en 48 heures, et il y a 15 000 dinars. Il faut acheter vite, mais sans se tromper : un faux fournisseur, un prix gonflé ou une facture payée deux fois, ce sont des médicaments qui n'arrivent jamais.

Dans une vraie organisation, on ne laisse jamais une seule personne choisir le fournisseur, valider et payer. On sépare les rôles. EL JOKER fait la même chose avec 5 agents :

Lem3alem

`Le chef de projet`

Il lit la demande et en fait une mission propre : lieu, nombre de personnes, budget, délai, articles. S'il manque une info importante, il la demande au lieu de deviner.

Lemhaf

`L'acheteur · IA (Claude)`

Il compare les offres des fournisseurs et recommande la meilleure, en expliquant pourquoi. C'est le seul qui utilise l'IA. Il peut se tromper, donc il n'a aucun pouvoir.

Nasnes

`Le contrôleur · code`

Il vérifie tout, de façon indépendante : le fournisseur existe et il est vérifié, il livre dans la région, les calculs sont justes, les documents sont là, la facture n'a jamais servi.

3ases

`Le règlement · code, sans IA`

Il applique 13 règles fixes et donne une décision : approuvé, bloqué, ou « un humain doit décider ». Personne ne peut le convaincre, parce que ce n'est pas une IA.

Kanawita

`Le caissier · code + Hedera`

Il paie, mais seulement si tout le monde a dit oui. Il revérifie lui-même avant de payer, et il refuse de payer deux fois. Au-dessus de la limite, il ne peut que **bloquer** l'argent dans le coffre.

Hedera

`Le notaire public`

Chaque décision importante y est enregistrée sous forme d'empreinte. Après, personne ne peut réécrire l'histoire sans que ça se voie. Et pour les gros montants, c'est Hedera qui exige la signature humaine.

Le coffre 2 sur 2

`Le coffre-fort à deux clés`

Un compte Hedera qui a besoin de **deux** signatures : celle des agents et celle d'un humain. Les agents peuvent préparer le paiement, mais seul l'humain peut le débloquer. S'il refuse, le paiement préparé est supprimé.

Deux organisations

`ONG et entreprise`

Les mêmes agents travaillent pour une ONG (limite 15 000 TND, eau et médicaments) et pour une entreprise, ACME (limite 6 000 TND, ordinateurs et fournitures). Seul le fichier de règles change.

CHAPITRE 2 · LE PLUS IMPORTANT

Ce qui se passe quand tu cliques « Envoyer »

Choisis un scénario, puis avance message par message. À gauche, la conversation telle qu'elle apparaît sur le site. À droite, l'explication de chaque message et le panneau « Live Hedera proofs » qui se remplit. Les textes en anglais sont exactement ceux du code.

Scénario A · Approuvé

You · Operator

Toi

500 people in Gabès need drinking water and essential medicine after a water network failure. Budget: 15,000 TND. Delivery within 48 hours to the civil-protection warehouse.

REQUEST SENT

ÉTAPE 1 · TOI (OPÉRATEUR)

Tu envoies la demande

- Le navigateur envoie une requête à l'API : `POST /api/scenarios/A_APPROVED/run` si tu envoies le scénario tel quel, ou `POST /api/missions` si tu as modifié le texte.
- Next.js (port 3000) transmet la requête à l'API Express (port 4000). Les clés secrètes ne sont jamais dans le navigateur.
- L'API crée une mission avec un identifiant `msn_…` et enregistre ton message dans la conversation.

Fichiers : **apps/web/components/ChatComposer.tsx** (bouton Envoyer), **apps/api/src/app.ts** (routes), **orchestrator/missionService.ts** (createMission).

L

Lem3alem COORDINATOR

Team, we have a new mission for South Tunisia Relief Network (demo NGO): Emergency water and essential medicine for 500 people in Gabès after a water network failure. Budget 15,000 TND, 500 people, Gabès.

`MISSION CREATED` `RELIEF NGO` `DETERMINISTIC`

ÉTAPE 2 · LEM3ALEM

Lem3alem crée la mission

- Il vérifie que la mission est complète : lieu, bénéficiaires, articles, budget, délai. Sinon il répond « Some information is missing » et s'arrête.
- Badge DETERMINISTIC : le scénario était déjà structuré, Claude n'a pas été appelé. Avec un texte libre, tu verrais « REAL AI · claude-opus-5-5 ».
- **Première preuve** : l'événement MISSION_CREATED est créé, son empreinte calculée, et il part vers Hedera (à droite : « consensus… »).

Fichiers :

agents/lem3alem.ts

(planMission) et

audit/proofService.ts

(recordEvent : empreinte + chaîne + envoi HCS en file d'attente).

H

Lemhaf AI RESEARCH

Searching… asking Claude to rank supplier offers for Gabès.

`AI ANALYZING`

ÉTAPE 3 · LEMHAF

Lemhaf commence la recherche

- La mission passe au statut RESEARCHING.
- Le « marché de démo » (fournisseurs fictifs) produit les offres des fournisseurs qui livrent dans cette région.
- Ces offres, avec leurs totaux déjà calculés, sont envoyées à Claude par internet.

Fichiers :

agents/marketplace.ts

(offres) puis

agents/lemhaf.ts

→

ai/aiClient.ts

(appel à Claude, réponse JSON validée par Zod). Si Claude n'est pas configuré, le texte dit « rule-based fallback ».

H

Lemhaf AI RESEARCH

I found 3 offers. My pick: Oasis Medical & Water Supply (demo) — 12,450 TND, 20h delivery. [explication de Claude]

Ranking of 3 offers (cliquable sur le site) : rang, score et justification pour chaque offre, plus les « Concerns » de Claude.

`RECOMMENDATION`

`REAL AI · claude-opus-5-5`

`DEMO DATA`

ÉTAPE 4 · LEMHAF

Claude recommande Oasis

- Claude a classé 3 offres et choisi Oasis : 3 000 bouteilles × 1,5 TND + 50 kits × 126 TND + 1 000 sachets × 1,65 TND = 12 450 TND.
- REAL AI · claude-opus-5-5 = c'est vraiment Claude qui a répondu. DEMO DATA = les fournisseurs sont fictifs.
- **Preuve 2** : AI_RECOMMENDATION. On garde aussi le mode (REAL_AI ou FALLBACK) et le modèle, pour prouver plus tard qui a recommandé quoi.

Le texte de Claude est une recommandation, pas un ordre. Il est enregistré dans la base (table des recommandations) et sera contrôlé par Nasnes.

N

Nasnes VERIFIER

Checking Oasis Medical & Water Supply (demo) against the registry, the evidence and LEMHAF's claims.

`VERIFYING`

ÉTAPE 5 · NASNES

Nasnes commence la vérification

- Statut VERIFYING.
- Nasnes ne fait pas confiance à Lemhaf : il relit lui-même le registre des fournisseurs, l'offre, les documents et l'historique des factures.

Fichier :

agents/nasnes.ts

. Code pur, aucune IA (badge DETERMINISTIC).

N

Nasnes VERIFIER

All good. Oasis Medical & Water Supply (demo) checks out: registry, region, items, arithmetic and evidence all verified.

- ✓ **Supplier in registry** — Oasis Medical & Water Supply (demo) found in registry
- ✓ **Supplier verified** — Verified — registre de démo
- ✓ **Serves mission region** — Serves Gabès
- ✓ **All requested items present** — 3/3 items in full quantity
- ✓ **Price arithmetic consistent** — Line totals sum to 12,450 TND
- ✓ **Evidence documents** — Quote, invoice and license evidence present (hashed)
- ✓ **Invoice not reused** — Invoice … is new
- ✓ **Recommendation claims supported** — Stated facts match the records
- ✓ **Plausible pricing** — Unit prices plausible vs reference
- ✓ **Delivery meets deadline** — 20h fits the deadline

`VERIFIED` `DETERMINISTIC`

ÉTAPE 6 · NASNES

Nasnes valide : 10 contrôles sur 10

- Les 7 premiers contrôles sont bloquants (REJECT) s'ils échouent. Les 3 derniers (affirmations de l'IA, prix plausibles, délai) demandent une revue (REVIEW).
- « Recommendation claims supported » : il compare ce que Claude affirme (nom, prix, délai, « fournisseur vérifié ») avec la base. Si Claude a inventé, ça se voit ici.
- **Preuve 3** : SUPPLIER_VERIFIED, avec les empreintes des documents (devis, facture, licence).

Résultat possible : VERIFIED, NEEDS_REVIEW ou REJECTED. Kanawita exigera VERIFIED.

3

3ases GUARDIAN

Rules are rules. Applying policy 2.1.0+... — deterministic rules, no AI.

`EVALUATING`

ÉTAPE 7 · 3ASES

3ases applique le règlement

- Statut EVALUATING.
- « 2.1.0+... » est la version des règles : 2.1.0 plus une empreinte du fichier policy.json. Si quelqu'un change une règle, la version change, et elle est écrite dans chaque preuve.
- 3ases reconstruit une « photo » des faits à partir de la base de données et calcule son empreinte (inputHash).

Fichiers : **agents/guardian/engine.ts** (fonction **evaluate**) et **agents/guardian/policy.json** (limite autonome 15 000 TND, plafond 75 000 TND, régions et catégories autorisées, seuils de prix).

3

3ases GUARDIAN

Everything checks out. APPROVED — risk LOW. All 12 policy checks passed.

- ✓ **Valid mission state** — PASS
- ✓ **NASNES verification** — PASS
- ✓ **Verified supplier** — PASS
- ✓ **Allowed region** — PASS
- ✓ **Allowed spending category** — PASS
- ✓ **Currency match** — PASS
- ✓ **Within mission budget** — PASS
- ✓ **Price anomaly** — PASS
- ✓ **Duplicate invoice** — PASS
- ✓ **Duplicate payment** — PASS
- ✓ **Autonomous spending limit** — PASS
- ✓ **Human approval threshold** — PASS

`APPROVED` `RISK LOW` `NO LLM`

ÉTAPE 8 · 3ASES

3ases approuve

- Le code contient 13 règles. Ici, 12 s'affichent : la 13ᵉ (« Offer under evaluation ») n'apparaît que s'il n'y a pas d'offre.
- Chaque règle donne PASS, REVIEW (un humain doit décider) ou BLOCK. Une seule BLOCK = mission bloquée.
- Risque LOW : un score calculé avec des poids (au-dessus de la limite autonome +40, proche du budget +10, alerte fournisseur +15, prix anormal +20, urgence critique +10). LOW < 20 ≤ MEDIUM < 40 ≤ HIGH < 70 ≤ CRITICAL.
- **Preuve 4** : GUARDIAN_APPROVED. C'est l'autorisation de payer, et Kanawita attendra qu'elle soit sur Hedera.

Sur le site, sous le message : la version des règles, le score de risque et l'

input hash

(empreinte des faits évalués).

K

Kanawita PAYMENT

Checking authorization before moving any money...

`CHECKING AUTHORIZATION`

ÉTAPE 9 · KANAWITA

Kanawita revérifie tout

- Il ne fait confiance à personne. Avant de payer, il vérifie lui-même : 1) il relance 3ases sur des données fraîches (doit dire APPROVED), 2) Nasnes = VERIFIED, 3) une validation humaine valide si elle était exigée, 4) la mission n'est pas déjà payée et la clé d'idempotence jamais utilisée.
- 5) Il attend que la preuve GUARDIAN_APPROVED soit vraiment sur Hedera (jusqu'à 45 s). Pas de preuve = pas de paiement.

Fichier :

agents/kanawita.ts

(authorize + execute). Clé d'idempotence = un identifiant unique de ce paiement ; la base refuse un deuxième paiement avec la même clé.

K

Kanawita PAYMENT

Authorized. Sending 12,450 EJTND on Hedera testnet…

`PAYING`

ÉTAPE 10 · KANAWITA

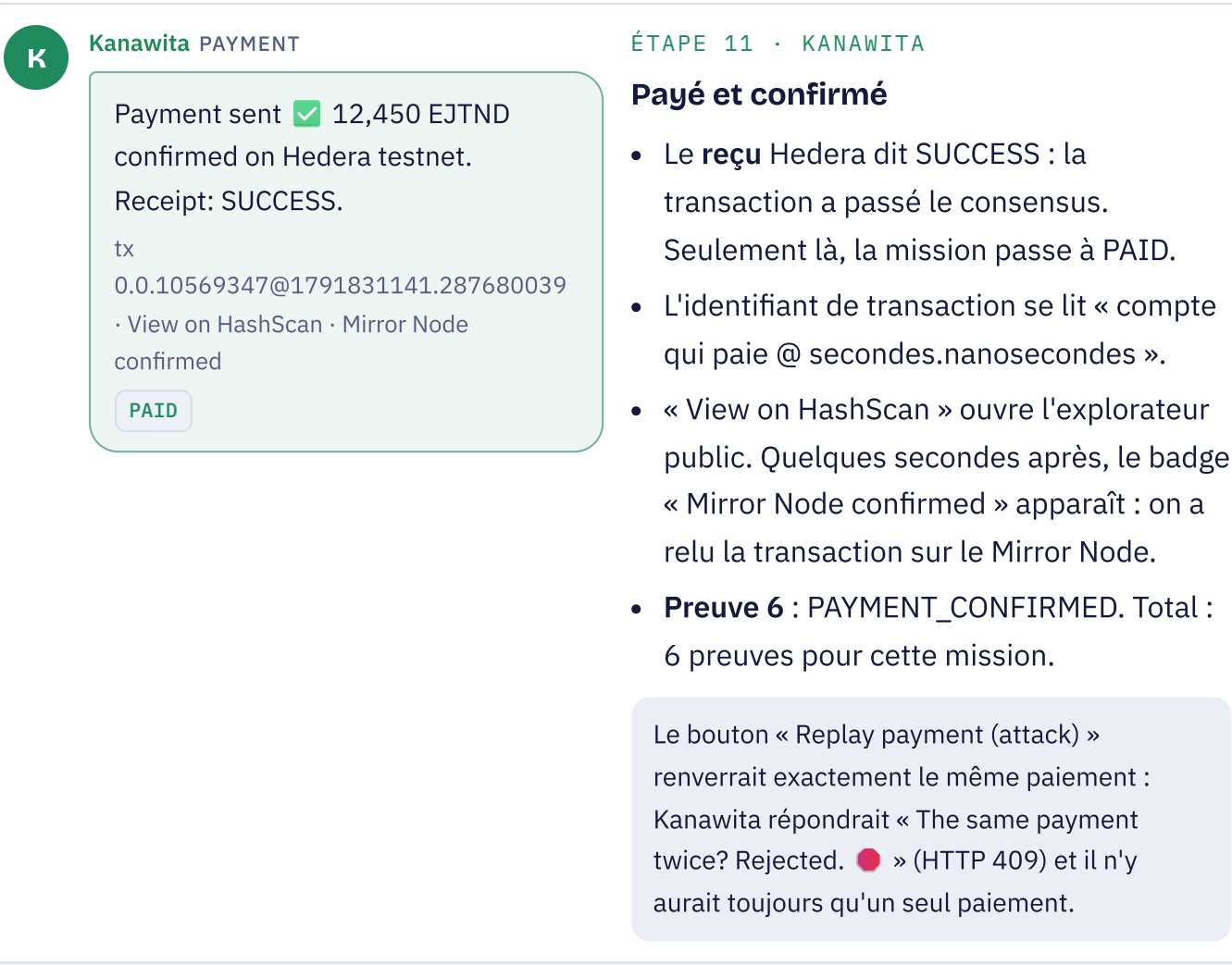
Le paiement part

- Avec le jeton EJTND (notre dinar de test, créé avec le Token Service de Hedera : 1 EJTND = 1 dinar), on paie le montant complet : 12 450 EJTND. Sans ce jeton, on paie un petit montant en HBAR : 12 450 TND ÷ 1 000 × 0,08 = 0,996 $\hbar$.
- Le paiement est d'abord inscrit dans la base (statut INITIATED) : les contraintes de la base refusent un doublon.
- **Preuve 5** : PAYMENT_INITIATED.
- Puis une **TransferTransaction** Hedera (transfert du jeton EJTND) est signée avec la clé des agents et envoyée au compte du fournisseur de démo. 12 450 TND est sous la limite de 15 000 TND : les agents paient seuls, depuis leur propre compte.

Fichier :

hedera/sdkGateway.ts

(SDK @hashgraph/sdk). Si le reçu se perd, on redemande le reçu de la même transaction au lieu d'en renvoyer une nouvelle.



ÉTAPE 11 · KANAWITA

Payé et confirmé

- Le **reçu** Hedera dit SUCCESS : la transaction a passé le consensus. Seulement là, la mission passe à PAID.
- L'identifiant de transaction se lit « compte qui paie @ secondes.nanosecondes ».
- « View on HashScan » ouvre l'explorateur public. Quelques secondes après, le badge « Mirror Node confirmed » apparaît : on a relu la transaction sur le Mirror Node.
- **Preuve 6** : PAYMENT_CONFIRMED. Total : 6 preuves pour cette mission.

Le bouton « Replay payment (attack) » renverrait exactement le même paiement : Kanawita répondrait « The same payment twice? Rejected. 🛑 » (HTTP 409) et il n'y aurait toujours qu'un seul paiement.

N

Nasnes VERIFIER

Proof verified: [résumé de la vérification].

PROOF VERIFIED

BONUS · NASNES

Bonus : tu cliques « Verify against Hedera »

- L'API relit sur le **Mirror Node** tous les messages du topic de cette mission.
- Pour chaque événement local, elle recalcule l'empreinte et la compare à la copie sur Hedera. Elle vérifie aussi qu'aucun événement n'a disparu.
- Tout correspond : PROOF VERIFIED. Si quelqu'un avait modifié la base : « Someone changed the history! » et TAMPERING DETECTED (voir chapitre 6).

Fichiers : **audit/proofService.ts** (verifyMissionTrail) et **hedera/mirror.ts** .

Scénario D · Fournisseur piège

You · Operator

Rush: 500 people in Gabès need drinking water and essential medicine within 12 hours. Budget: 15,000 TND.

REQUEST SENT

Toi

ÉTAPE 1 · TOI (OPÉRATEUR)

Une demande très urgente

- Même lieu, même budget, mais seulement 12 heures. Le piège : le seul fournisseur assez rapide n'est pas vérifié.

Route : `POST /api/scenarios/D_UNVERIFIED_SUPPLIER/run` .

L

Lem3alem COORDINATOR

Team, we have a new mission for South Tunisia Relief Network (demo NGO): ... Budget 15,000 TND, 500 people, Gabès.

`MISSION CREATED` `RELIEF NGO` `DETERMINISTIC`

ÉTAPE 2 · LEM3ALEM

Mission créée

- Comme dans le scénario A. Preuve 1 : MISSION_CREATED.

H

Lemhaf AI RESEARCH

Searching... asking Claude to rank supplier offers for Gabès.

`AI ANALYZING`

ÉTAPE 3 · LEMHAF

Lemhaf commence la recherche

- La mission passe au statut RESEARCHING.
- Le « marché de démo » (fournisseurs fictifs) produit les offres des fournisseurs qui livrent dans cette région.
- Ces offres, avec leurs totaux déjà calculés, sont envoyées à Claude par internet.

Fichiers :

agents/marketplace.ts

(offres) puis

agents/lemhaf.ts

→

ai/aiClient.ts

(appel à Claude, réponse JSON validée par Zod). Si Claude n'est pas configuré, le texte dit « rule-based fallback ».

H

Lemhaf AI RESEARCH

My pick: Sahel Express Trading (demo) — 6,810 TND, 10h delivery. [explication de Claude, avec un avertissement sur les prix]

`RECOMMENDATION`

`REAL AI · claude-opus-5-5`

`DEMO DATA`

ÉTAPE 4 · LEMHAF

L'IA tombe dans le piège

- Sahel livre en 10 h et coûte deux fois moins cher. Sous pression, Claude le choisit.
- Lors de notre test réel, Claude a même signalé un risque de fraude dans ses « Concerns ». Mais il recommande quand même : c'est son rôle, il n'a pas le pouvoir de décider.
- Preuve 2 : AI_RECOMMENDATION (la mauvaise recommandation est elle aussi prouvée).

On ne montre pas à Claude le statut « vérifié » des fournisseurs : il ne peut pas tricher en le recopiant.

N

Nasnes VERIFIER

Checking Sahel Express Trading (demo) against the registry, the evidence and LEMHAF's claims.

`VERIFYING`

ÉTAPE 5 · NASNES

Nasnes commence la vérification

- Statut VERIFYING.
- Nasnes ne fait pas confiance à Lemhaf : il relit lui-même le registre des fournisseurs, l'offre, les documents et l'historique des factures.

Fichier : **agents/nasnes.ts** . Code pur, aucune IA (badge DETERMINISTIC).

N

Nasnes VERIFIER

Stop! REJECTED: Not verified (flags: NOT_IN_REGISTRY, NO_LICENSE_EVIDENCE, NEW_COUNTERPARTY) · …

✕ **Supplier verified** — Not verified (flags: …)

✕ **Evidence documents** — Missing: license

! **Plausible pricing** — Suspiciously low: …

✓ **Serves mission region** — Serves Gabès

✓ **Delivery meets deadline** — 10h fits the deadline

`REJECTED` `DETERMINISTIC`

ÉTAPE 6 · NASNES

Nasnes rejette

- Le fournisseur n'est pas vérifié, il manque la licence, et les prix sont anormalement bas (moins de 40 % sous la référence).
- Résultat REJECTED. La liste ci-dessus est un extrait : sur le site, les 10 contrôles s'affichent avec ✓, ✕ ou !.
- Preuve 3 : SUPPLIER_REJECTED (en rouge dans le panneau de droite).

3

3ases GUARDIAN

Rules are rules. Applying policy 2.1.0+… — deterministic rules, no AI.

`EVALUATING`

ÉTAPE 7 · 3ASES

3ases applique le règlement

- Statut EVALUATING.
- « 2.1.0+… » est la version des règles : 2.1.0 plus une empreinte du fichier policy.json. Si quelqu'un change une règle, la version change, et elle est écrite dans chaque preuve.
- 3ases reconstruit une « photo » des faits à partir de la base de données et calcule son empreinte (inputHash).

Fichiers : **agents/guardian/engine.ts** (fonction **evaluate**) et **agents/guardian/policy.json** (limite autonome 15 000 TND, plafond 75 000 TND, régions et catégories autorisées, seuils de prix).

3 **3ases** GUARDIAN

Stop! BLOCKED — NASNES result is REJECTED; payment requires VERIFIED · ...

BLOCKED | RISK CRITICAL | NO LLM

ÉTAPE 8 · 3ASES

3ases bloque

- Au moins une règle BLOCK : la mission est bloquée. Le risque passe automatiquement à CRITICAL.
- Aucun humain ne peut débloquer une mission BLOCKED : la validation humaine ne sert que pour les cas REVIEW.
- Preuve 4 : GUARDIAN_BLOCKED. Le refus lui-même est prouvé sur Hedera.

K **Kanawita** PAYMENT

I won't move a single cent. The Guardian said no, so no payment.

NO PAYMENT

ÉTAPE 9 · KANAWITA

Aucun paiement

- Kanawita n'est même pas lancé pour payer. Statut final : BLOCKED. 4 preuves au total.

Scénario B · Budget trop petit

You · Operator

500 people in Gabès need drinking water and essential medicine. We only have 300 TND available. Delivery within 48 hours.

REQUEST SENT

Toi

ÉTAPE 1 · TOI (OPÉRATEUR)

Un budget de 300 TND

- Même besoin que A, mais seulement 300 dinars.

L **Lem3alem** COORDINATOR

Team, we have a new mission for South Tunisia Relief Network (demo NGO): Emergency water and essential medicine for 500 people in Gabès with a 300 TND budget. Budget 300 TND, 500 people, Gabès.

MISSION CREATED | RELIEF NGO | DETERMINISTIC

ÉTAPE 2 · LEM3ALEM

Mission créée

- Preuve 1.

H

Lemhaf AI RESEARCH

Searching… asking Claude to rank supplier offers for Gabès.

AI ANALYZING

ÉTAPE 3 · LEMHAF

Lemhaf commence la recherche

- La mission passe au statut RESEARCHING.
- Le « marché de démo » (fournisseurs fictifs) produit les offres des fournisseurs qui livrent dans cette région.
- Ces offres, avec leurs totaux déjà calculés, sont envoyées à Claude par internet.

Fichiers :

agents/marketplace.ts

(offres) puis

agents/lemhaf.ts

→

ai/aiClient.ts

(appel à Claude, réponse JSON validée par Zod). Si Claude n'est pas configuré, le texte dit « rule-based fallback ».

H

Lemhaf AI RESEARCH

I found 3 offers. My pick: Oasis Medical & Water Supply (demo) — 12,450 TND, 20h delivery. …

RECOMMENDATION

REAL AI · claude-opus-5-5

DEMO DATA

ÉTAPE 4 · LEMHAF

Claude recommande la meilleure offre

- Le budget n'est pas le problème de Lemhaf : il recommande la meilleure offre. Preuve 2.

N

Nasnes VERIFIER

Checking Oasis Medical & Water Supply (demo) against the registry, the evidence and LEMHAF's claims.

`VERIFYING`

ÉTAPE 5 · NASNES

Nasnes commence la vérification

- Statut VERIFYING.
- Nasnes ne fait pas confiance à Lemhaf : il relit lui-même le registre des fournisseurs, l'offre, les documents et l'historique des factures.

Fichier : **agents/nasnes.ts** . Code pur, aucune IA (badge DETERMINISTIC).

N

Nasnes VERIFIER

All good. Oasis Medical & Water Supply (demo) checks out: registry, region, items, arithmetic and evidence all verified.

`VERIFIED` `DETERMINISTIC`

ÉTAPE 6 · NASNES

Le fournisseur est bon

- Nasnes contrôle le fournisseur et l'offre, pas le budget. Tout est correct. Preuve 3.

3 **3ases** GUARDIAN

Rules are rules. Applying policy 2.1.0+... — deterministic rules, no AI.

`EVALUATING`

ÉTAPE 7 · 3ASES

3ases applique le règlement

- Statut EVALUATING.
- « 2.1.0+... » est la version des règles : 2.1.0 plus une empreinte du fichier policy.json. Si quelqu'un change une règle, la version change, et elle est écrite dans chaque preuve.
- 3ases reconstruit une « photo » des faits à partir de la base de données et calcule son empreinte (inputHash).

Fichiers : **agents/guardian/engine.ts** (fonction **evaluate**) et **agents/guardian/policy.json** (limite autonome 15 000 TND, plafond 75 000 TND, régions et catégories autorisées, seuils de prix).

3 **3ases** GUARDIAN

Stop! BLOCKED — Offer 12,450 TND exceeds budget 300 TND by 12,150 TND

`BLOCKED` `RISK CRITICAL` `NO LLM`

ÉTAPE 8 · 3ASES

La règle du budget bloque

- Règle « Within mission budget » = BLOCK. Chaque acteur a son rôle : c'est 3ases qui fait respecter l'argent disponible.
- Preuve 4 : GUARDIAN_BLOCKED.

K **Kanawita** PAYMENT

I won't move a single cent. The Guardian said no, so no payment.

`NO PAYMENT`

ÉTAPE 9 · KANAWITA

Aucun paiement

- L'argent ne bouge pas, et le refus est prouvé sur Hedera.

Scénario E · Coffre Hedera

You · Operator

2,000 displaced people in Médenine need drinking water and essential medicine. Budget: 60,000 TND. Delivery within 72 hours.

REQUEST SENT

Toi

ÉTAPE 1 · TOI (OPÉRATEUR)

Une grosse mission

- 2 000 personnes, 60 000 TND de budget. La meilleure offre dépassera la limite autonome de 15 000 TND.

L

Lem3alem COORDINATOR

Team, we have a new mission for South Tunisia Relief Network (demo NGO): … Budget 60,000 TND, 2,000 people, Médenine.

MISSION CREATED RELIEF NGO DETERMINISTIC

ÉTAPE 2 · LEM3ALEM

Mission créée

- Preuve 1.

H

Lemhaf AI RESEARCH

Searching… asking Claude to rank supplier offers for Médenine.

AI ANALYZING

ÉTAPE 3 · LEMHAF

Lemhaf commence la recherche

- La mission passe au statut RESEARCHING.
- Le « marché de démo » (fournisseurs fictifs) produit les offres des fournisseurs qui livrent dans cette région.
- Ces offres, avec leurs totaux déjà calculés, sont envoyées à Claude par internet.

Fichiers :

agents/marketplace.ts

(offres) puis

agents/lemhaf.ts

→

ai/aiClient.ts

(appel à Claude, réponse JSON validée par Zod). Si Claude n'est pas configuré, le texte dit « rule-based fallback ».

H

Lemhaf AI RESEARCH

My pick: Oasis Medical & Water Supply (demo) — 49,800 TND, 20h delivery. ...

RECOMMENDATION

REAL AI · claude-opus-5-5

DEMO DATA

ÉTAPE 4 · LEMHAF

Offre à 49 800 TND

- 12 000 bouteilles × 1,5 TND + 200 kits × 126 TND + 4 000 sachets × 1,65 TND = 49 800 TND. Preuve 2.

N

Nasnes VERIFIER

Checking Oasis Medical & Water Supply (demo) against the registry, the evidence and LEMHAF's claims.

VERIFYING

ÉTAPE 5 · NASNES

Nasnes commence la vérification

- Statut VERIFYING.
- Nasnes ne fait pas confiance à Lemhaf : il relit lui-même le registre des fournisseurs, l'offre, les documents et l'historique des factures.

Fichier : **agents/nasnes.ts** . Code pur, aucune IA (badge DETERMINISTIC).

N

Nasnes VERIFIER

All good. Oasis Medical & Water Supply (demo) checks out: registry, region, items, arithmetic and evidence all verified.

VERIFIED DETERMINISTIC

ÉTAPE 6 · NASNES

Fournisseur vérifié

- Preuve 3.

3 **3ases** GUARDIAN

Rules are rules. Applying policy 2.1.0+... — deterministic rules, no AI.

`EVALUATING`

ÉTAPE 7 · 3ASES

3ases applique le règlement

- Statut EVALUATING.
- « 2.1.0+... » est la version des règles : 2.1.0 plus une empreinte du fichier policy.json. Si quelqu'un change une règle, la version change, et elle est écrite dans chaque preuve.
- 3ases reconstruit une « photo » des faits à partir de la base de données et calcule son empreinte (inputHash).

Fichiers : **agents/guardian/engine.ts** (fonction **evaluate**) et **agents/guardian/policy.json** (limite autonome 15 000 TND, plafond 75 000 TND, régions et catégories autorisées, seuils de prix).

3 **3ases** GUARDIAN

This is above my autonomous limit. HUMAN REVIEW REQUIRED — 49,800 TND exceeds the autonomous limit 15,000 TND — a human must decide · ...

`HUMAN REVIEW` `RISK HIGH` `NO LLM`

ÉTAPE 8 · 3ASES

3ases demande un humain

- Aucune règle BLOCK, mais la règle « Autonomous spending limit » dit REVIEW. Le risque monte à HIGH (+40 car au-dessus de la limite).
- 3ases n'approuve pas et ne bloque pas : il escalade. Preuve 4 : HUMAN_REVIEW_REQUIRED.
- Il retient l'**input hash** : l'empreinte exacte des faits que l'humain va voir.

K

Kanawita PAYMENT

Payment locked on Hedera: schedule 0.0.... holds 49,800 EJTND from the vault 0.0..... My key is 1 of the 2 signatures it needs — I cannot release it alone.

`LOCKED ON HEDERA`

`1 OF 2 SIGNATURES`

ÉTAPE 9 · KANAWITA

Kanawita bloque l'argent dans le coffre

- Au-dessus de la limite, l'argent n'est pas sur le compte des agents : il est dans un **coffre** dont la clé est une « clé à seuil » 2 sur 2 (la clé des agents + la clé d'un approbateur humain).
- Kanawita crée une **transaction programmée** (schedule) Hedera : le transfert coffre → fournisseur, déjà signé par les agents. Il manque une signature, donc rien ne bouge.
- Preuve 5 : PAYMENT_SCHEDULED, avec le numéro de la transaction programmée. Sur le site, la carte d'approbation affiche « Locked on Hedera · 1 of 2 signatures » et un lien HashScan.

Fichier : **agents/kanawita.ts** (lockInVault) → **hedera/sdkGateway.ts** : **ScheduleCreateTransaction**. Cette réservation ne change pas les faits que l'humain valide (l'input hash reste le même).

You · Operator

[opérateur]: APPROVED — “[ta raison]”

HUMAN APPROVED DEMO OPERATOR

Toi

ÉTAPE 10 · TOI (OPÉRATEUR)

Tu approuves

- Avant d'accepter, l'API recalcule l'empreinte des faits. Si quelque chose a changé depuis la demande de revue : refus « STALE_REVIEW ».
- Preuve 6 : HUMAN_APPROVED, liée à l'input hash. DEMO OPERATOR rappelle que ce n'est pas un vrai système d'identité.

Route :

`POST /api/missions/:id/approvals`

.

3

3ases GUARDIAN

Everything checks out. APPROVED — risk HIGH. All 12 policy checks passed (with human approval).

APPROVED RISK HIGH NO LLM

ÉTAPE 11 · 3ASES

3ases réévalue avec l'approbation

- Avec une approbation humaine valide, les règles REVIEW passent. Preuve 7 : GUARDIAN_APPROVED.

K

Kanawita PAYMENT

Approved. Adding the approver's signature to schedule 0.0.... on Hedera...

CO-SIGNING

ÉTAPE 12 · KANAWITA

La deuxième signature

- Kanawita revérifie tout (comme toujours), puis ajoute la signature de l'approbateur à la transaction programmée : **ScheduleSignTransaction**.
- Preuve 8 : PAYMENT_INITIATED.

Dans la démo, la clé de l'approbateur est gardée par notre serveur et utilisée seulement après opérateur + PIN. En vrai, elle serait dans le portefeuille de l'approbateur.

K

Kanawita PAYMENT

Payment released ✅ The approver co-signed and Hedera executed schedule 0.0....: 49,800 EJTND paid from the vault. Receipt: SUCCESS.

PAID CO-SIGNED ON HEDERA

ÉTAPE 13 · KANAWITA

Hedera exécute le paiement

- Avec 2 signatures sur 2, c'est **Hedera lui-même** qui exécute le transfert, dans le même tour de consensus.
- Preuve 9 : PAYMENT_CONFIRMED (« 2 of 2 »). Sur HashScan, la transaction programmée apparaît « executed » avec 2 signatures.
- Si l'opérateur avait refusé : la transaction programmée est supprimée (preuve PAYMENT_CANCELLED) et l'argent ne quitte jamais le coffre.

Même quelqu'un qui volerait la clé de notre serveur ne pourrait pas vider le coffre : il lui manquerait la clé de l'approbateur.

Scénario I · ACME (entreprise)

You · Operator

ACME Tunisia: 12 laptops and 12 monitors for the new engineering team in Tunis. Budget: 45,000 TND. Delivery within 7 days.

REQUEST SENT

Toi

ÉTAPE 1 · TOI (OPÉRATEUR)

Une entreprise, pas une ONG

- Même application, même agents, mais une autre organisation : ACME Tunisia (fictive), qui équipe une nouvelle équipe.
- La demande cite « ACME » : la mission est rattachée à ACME, donc à **sa** politique de dépenses.

Route : `POST /api/scenarios/I_ACME_LAPTOPS/run` .

L

Lem3alem COORDINATOR

Team, we have a new mission for ACME Tunisia (demo company): Laptops and monitors for ACME Tunisia's new engineering team (12 employees). Budget 45,000 TND, 12 employees, Tunis.

`MISSION CREATED` `ACME` `DETERMINISTIC`

ÉTAPE 2 · LEM3ALEM

Mission créée pour ACME

- Les personnes s'appellent ici « employees » au lieu de « people ».
- Preuve 1 : MISSION_CREATED, avec la politique d'ACME (version 1.1.0) dans la preuve.

Fichier :

agents/guardian/policy.acme.json

: limite autonome 6 000 TND, plafond 150 000 TND, catégories ELECTRONICS et OFFICE, villes Tunis, Sousse, Sfax.

H

Lemhaf AI RESEARCH

Searching... asking Claude to rank supplier offers for Tunis.

`AI ANALYZING`

ÉTAPE 3 · LEMHAF

Lemhaf commence la recherche

- La mission passe au statut RESEARCHING.
- Le « marché de démo » (fournisseurs fictifs) produit les offres des fournisseurs qui livrent dans cette région.
- Ces offres, avec leurs totaux déjà calculés, sont envoyées à Claude par internet.

Fichiers :

agents/marketplace.ts

(offres) puis

agents/lemhaf.ts

→

ai/aiClient.ts

(appel à Claude, réponse JSON validée par Zod). Si Claude n'est pas configuré, le texte dit « rule-based fallback ».

H

Lemhaf AI RESEARCH

I found 2 offers. My pick: Carthage IT Distribution (demo) — 35,460 TND, 48h delivery. ...

RECOMMENDATION

REAL AI · claude-opus-5-5

DEMO DATA

ÉTAPE 4 · LEMHAF

Claude recommande Carthage IT

- 12 ordinateurs × 2 460 TND + 12 écrans × 495 TND = 35 460 TND. L'autre offre (Sousse Office Pro) coûte 36 684 TND.
- Preuve 2.

N

Nasnes VERIFIER

Checking Carthage IT Distribution (demo) against the registry, the evidence and LEMHAF's claims.

VERIFYING

ÉTAPE 5 · NASNES

Nasnes commence la vérification

- Statut VERIFYING.
- Nasnes ne fait pas confiance à Lemhaf : il relit lui-même le registre des fournisseurs, l'offre, les documents et l'historique des factures.

Fichier : **agents/nasnes.ts** . Code pur, aucune IA (badge DETERMINISTIC).

N

Nasnes VERIFIER

All good. Carthage IT Distribution (demo) checks out: registry, region, items, arithmetic and evidence all verified.

VERIFIED DETERMINISTIC

ÉTAPE 6 · NASNES

Fournisseur vérifié

- Les mêmes 10 contrôles que pour l'ONG. Preuve 3.

3ases GUARDIAN

Rules are rules. Applying policy 1.1.0+... — deterministic rules, no AI.

EVALUATING

ÉTAPE 7 · 3ASES

3ases applique les règles d'ACME

- Même code, autre fichier de règles : la version affichée est 1.1.0 (celle d'ACME), pas 2.1.0 (celle de l'ONG).

Fichier :

agents/guardian/policy.ts

(PolicyRegistry : une politique par organisation).

3ases GUARDIAN

This is above my autonomous limit. HUMAN REVIEW REQUIRED — Autonomous spending limit: 35,460 TND exceeds the autonomous limit 6,000 TND — a human must decide · ...

HUMAN REVIEW | RISK HIGH | NO LLM

ÉTAPE 8 · 3ASES

Au-dessus de la limite d'ACME

- 35 460 TND dépasse la limite de 6 000 TND d'ACME. Pour l'ONG, la limite serait 15 000 TND.
- Preuve 4.

Kanawita PAYMENT

Payment locked on Hedera: schedule 0.0.... holds 35,460 EJTND from the vault 0.0..... My key is 1 of the 2 signatures it needs — I cannot release it alone.

LOCKED ON HEDERA

1 OF 2 SIGNATURES

ÉTAPE 9 · KANAWITA

Bloqué dans le coffre

- Exactement le même mécanisme que pour l'ONG (scénario E) : une transaction programmée qui attend la signature humaine. Preuve 5.

You · Operator

ops-lead: APPROVED — “Budget line ENG-2026 approved by finance”

HUMAN APPROVED | DEMO OPERATOR

Toi

ÉTAPE 10 · TOI (OPÉRATEUR)

L'approbation

- Raison + PIN. Preuve 6.

K

Kanawita PAYMENT

Payment released ✅ The approver co-signed and Hedera executed schedule 0.0....: 35,460 EJTND paid from the vault. Receipt: SUCCESS.

`PAID` `CO-SIGNED ON HEDERA`

ÉTAPE 11 · KANAWITA

Payé par Hedera

- Avant ce message, tu vois aussi 3ases qui réévalue (APPROVED) et « Approved. Adding the approver's signature... ».
- Total : 9 preuves, sur le même topic Hedera que les missions de l'ONG.

C'est ce qui fait d'EL JOKER une plateforme : ajouter une organisation, c'est ajouter un fichier de règles.

Le chemin d'un clic, en une ligne

Ton navigateur envoie la demande au site (Next.js, port 3000), qui la transmet à l'API (Express, port 4000). L'API fait travailler les agents l'un après l'autre et enregistre chaque message dans la base SQLite. Pendant ce temps, ton navigateur redemande l'état de la mission toutes les 0,8 seconde (toutes les 3 secondes quand rien ne bouge) : c'est pour ça que les messages apparaissent un par un.

CHAPITRE 3

Tous les messages et badges, un par un

Les petits badges sous chaque message (et dans le panneau de droite) résument ce qui vient de se passer. Tape un mot pour filtrer.

Rechercher 65 / 65

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`AI · claude-opus-5-5`	Barre du haut	Claude est configuré : les recommandations sont faites par la vraie IA.
`AI · fallback`	Barre du haut	Pas de clé Anthropic : le classement se fait par règles simples. Tout le reste marche pareil.

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`HCS · 0.0.11007004`	Panneau des preuves	Le mode réel est actif : les preuves vont sur notre topic Hedera numéro 0.0.11007004 (testnet).
`Vault 2-of-2 · 0.0.…`	Barre du haut	Le coffre est configuré : au-dessus de la limite, l'argent ne sort qu'avec la signature humaine.
`Pays in EJTND (HTS)`	Barre du haut	Les paiements se font avec le jeton EJTND, notre dinar de test (montant complet). « Pays in HBAR (scaled) » = montant réduit en HBAR.
`Public verifier`	Barre du haut	Ouvre la page /verify : ton navigateur vérifie une mission tout seul, en lisant Hedera.
`RELIEF NGO / ACME`	Lem3alem, en-tête	L'organisation pour qui la mission est faite, donc les règles qui s'appliquent.
`LOCKED ON HEDERA`	Kanawita	Le paiement est bloqué dans le coffre comme transaction programmée Hedera. Il attend la signature humaine.
`LOCKED · SIMULATED`	Kanawita	Même chose avec le faux registre local (mode sans identifiants).
`1 OF 2 SIGNATURES`	Kanawita, carte	Les agents ont signé (1 sur 2). Il manque la signature de l'approbateur humain.
`CO-SIGNING`	Kanawita	L'approbateur signe la transaction programmée.
`CO-SIGNED ON HEDERA`	Kanawita	Les 2 signatures sont là : Hedera a exécuté le transfert lui-même.

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`SCHEDULE DELETED`	Kanawita	L'opérateur a refusé : la transaction programmée est supprimée. L'argent ne quitte pas le coffre.
`NEVER CO-SIGNED`	Kanawita	La suppression a échoué, mais sans la deuxième signature la transaction ne peut jamais s'exécuter, et elle expire.
`VAULT LOCK FAILED`	Kanawita	Hedera a refusé de créer la transaction programmée. Rien n'a bougé ; on réessaiera au moment de l'approbation.
`Locked in vault`	Carte Payment	Le paiement attend dans le coffre (1 signature sur 2).
`Payment locked in vault / Vault schedule deleted`	Panneau des preuves	Les preuves PAYMENT_SCHEDULED et PAYMENT_CANCELLED.
`Simulated ledger`	Panneau des preuves	Pas d'identifiants Hedera : un faux registre local remplace Hedera. Pas de lien HashScan.
`REQUEST SENT`	Ton message	Ta demande a été reçue par l'API et ouvre la conversation.
`MISSION CREATED`	Lem3alem	La demande est complète, la mission est créée. Première preuve envoyée vers Hedera.
`NEEDS INFORMATION`	Lem3alem	Il manque une info critique (lieu, budget…). Lem3alem refuse de deviner et liste ce qui manque.
`DETERMINISTIC`	Plusieurs agents	Ce résultat vient de code fixe, sans IA : mêmes données = même résultat, toujours.
`REAL AI · claude-opus-5-5`	Lem3alem, Lemhaf	Ce résultat vient vraiment de Claude.

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`FALLBACK · rule-based`	Lem3alem, Lemhaf	Claude n'a pas répondu (ou pas configuré) : classement par règles simples. La raison est indiquée.
`DEMO DATA`	Lemhaf	Les fournisseurs et offres sont fictifs (marqués « demo »).
`AI ANALYZING`	Lemhaf	Lemhaf attend la réponse de Claude (environ 10 secondes).
`RECOMMENDATION`	Lemhaf	La recommandation est prête. Le détail (Ranking of N offers) se déplie en cliquant.
`Concerns`	Détail de Lemhaf	Les inquiétudes que Claude signale lui-même (prix bizarre, délai serré…).
`NO OFFERS`	Lemhaf	Aucun fournisseur de démo ne livre cette région pour ces articles : mission bloquée.
`VERIFYING`	Nasnes	Nasnes est en train de faire ses 10 contrôles.
`VERIFIED`	Nasnes	Les 10 contrôles sont passés.
`NEEDS REVIEW`	Nasnes	Un contrôle « REVIEW » a échoué (affirmation de l'IA, prix suspect, délai) : pas de paiement sans revue.
`REJECTED`	Nasnes	Un contrôle bloquant a échoué (fournisseur, région, articles, calcul, documents, facture).
`✓ ✕ !`	Listes de contrôles	✓ réussi, ✕ échec bloquant, ! demande une revue humaine.
`EVALUATING`	3ases	3ases applique les règles de policy.json.

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`APPROVED`	3ases	Toutes les règles sont PASS (ou REVIEW couvertes par une approbation humaine valide).
`BLOCKED`	3ases	Au moins une règle BLOCK. Aucun humain ne peut passer outre.
`HUMAN REVIEW`	3ases	Pas de blocage, mais au moins une règle REVIEW (au-dessus de la limite, 15 000 TND pour l'ONG, ou risque HIGH) : un humain doit décider.
`RISK LOW / MEDIUM / HIGH / CRITICAL`	3ases	Niveau de risque calculé par points. Score < 20 LOW, ≥ 20 MEDIUM, ≥ 40 HIGH, ≥ 70 CRITICAL. Une mission bloquée est CRITICAL.
`NO LLM`	3ases	Rappel : 3ases n'utilise aucun modèle de langage (aucune IA).
`Policy 2.1.0+… · input hash`	Sous le message de 3ases	Version des règles (avec l'empreinte du fichier) et empreinte des faits évalués.
`CHECKING AUTHORIZATION`	Kanawita	Kanawita revérifie tout lui-même avant de payer.
`PAYING`	Kanawita	La transaction Hedera est envoyée, on attend le reçu.
`PAID`	Kanawita	Reçu Hedera SUCCESS : le paiement est confirmé sur le testnet.
`PAID · SIMULATED`	Kanawita	Mode simulé : aucun argent n'a bougé.
`View on HashScan`	Message de paiement	Ouvre la transaction sur HashScan, l'explorateur public de Hedera.

BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`Mirror Node confirmed`	Message de paiement	On a relu la transaction sur le Mirror Node : elle existe bien et a réussi.
`NO PAYMENT`	Kanawita	3ases a dit non : aucun paiement, Kanawita ne bouge pas.
`PAYMENT ON HOLD`	Kanawita	Le paiement attend la décision d'un opérateur.
`PAYMENT REFUSED`	Kanawita	Une condition manque au moment de payer (la liste des raisons est affichée).
`DUPLICATE BLOCKED`	Kanawita	Quelqu'un a renvoyé le même paiement : refusé (HTTP 409). Toujours un seul paiement.
`PAYMENT FAILED`	Kanawita	Hedera n'a pas confirmé : rien n'est marqué comme payé.
`HUMAN APPROVED / HUMAN REJECTED`	Ton message	Ta décision sur la carte d'approbation, enregistrée et prouvée.
`DEMO OPERATOR`	Ton message	Le système d'opérateur est une démo (PIN), pas une vraie identité.
`PROOF VERIFIED`	Nasnes	Après « Verify against Hedera » : la base locale correspond exactement aux preuves sur Hedera.
`TAMPERING DETECTED`	Nasnes	La base locale ne correspond plus à Hedera : quelqu'un a modifié ou effacé l'historique. L'alerte est elle-même prouvée.
`SIMULATED TAMPERING`	System	Tu as cliqué un bouton d'attaque de démo : la base a été modifiée exprès.

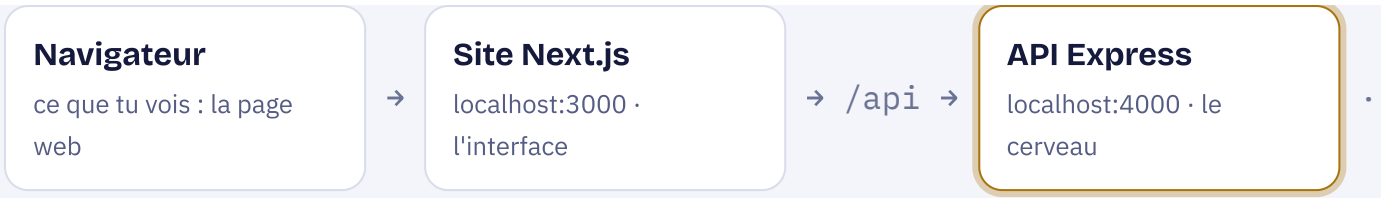
BADGE OU TEXTE	OÙ	CE QUE ÇA VEUT DIRE
`ERROR`	System	Erreur technique dans le pipeline. Aucun paiement n'a été fait.
`consensus…`	Panneau des preuves	La preuve est partie vers Hedera, on attend le consensus (quelques secondes).
`on Hedera`	Panneau des preuves	La preuve est ancrée : Hedera l'a acceptée et horodatée.
`simulated`	Panneau des preuves	Preuve enregistrée dans le faux registre local (mode sans identifiants).
`seq #12`	Panneau des preuves	Numéro d'ordre du message dans notre topic Hedera (toutes missions confondues).
`sha256 a3f9…`	Panneau des preuves	Le début de l'empreinte de la preuve complète (champ h du message Hedera).
`HashScan (preuve)`	Panneau des preuves	Ouvre ce message précis du topic sur HashScan.
`Mission created … Payment confirmed`	Panneau des preuves	Le type d'événement prouvé. En rouge : Blocked, Rejected, Failed, Tampering.

CHAPITRE 4

L'architecture : qui parle à qui

Clique sur une boîte pour savoir ce qu'elle fait, avec quelle technologie, et où elle se trouve dans le code. Tout le projet est écrit en TypeScript (du JavaScript avec des types).

SUR TON PC, DANS LE NAVIGATEUR



Vérificateur public
/verify · dans ton navigateur

DANS L'API

Les 5 agents
le pipeline de la mission

3ases + policy.json
les 13 règles

Service de preuves
empreintes + chaîne

Schémas partagés
Zod, packages/shared

CE QUE L'API UTILISE

SQLite
la base de données locale

API Claude
Anthropic, sur internet

Nœuds Hedera
testnet · écriture

Mirror Node
testnet · lecture

Navigateur

Ce que tu vois. Il ne contient aucune clé secrète.

- Affiche la page envoyée par Next.js.
- Envoie tes actions (Envoyer, Approve, Verify…) à `/api/…` sur le même site.
- Redemande l'état de la mission toutes les 0,8 s pendant le travail, 3 s sinon. Le serveur répond « rien de nouveau » (304) si rien n'a changé.

Site Next.js (apps/web)

L'interface : landing, conversation, panneaux de preuves.

- **Next.js 16** + **React 19** pour les composants, **Tailwind CSS 4** pour le style.
- Un « proxy » envoie tout ce qui commence par /api vers l'API sur le port 4000 (fichier `next.config.ts`).
- Composants clés : `ConversationPanel`, `ChatFeed` (messages), `ProofFeed` (preuves), `PipelineStrip` (les 5 agents en haut), `ApprovalCard` (PIN), `ProofVerify` (attaques + vérification).
- La logique de récupération est dans `lib/useMissionControl.ts`.

API Express (apps/api)

Le cerveau : c'est lui qui a les clés et qui parle à Claude et à Hedera.

- **Express 5** sur Node.js 22+, en TypeScript.
- Routes principales : créer une mission, lancer un scénario, lire l'état, approuver, rejouer un paiement, vérifier, simuler une attaque (fichier `app.ts`).
- Lit les secrets dans `.env` (clé Hedera, clé Anthropic). Le journal masque les valeurs qui ressemblent à des clés.

Les 5 agents

Le pipeline lancé pour chaque mission.

- `orchestrator/missionService.ts` enchaîne : Lem3alem → Lemhaf → Nasnes → 3ases → (humain) → Kanawita.
- Chaque agent est un fichier dans `agents/`.
- Chaque étape écrit un message de conversation (pour toi) et, si elle est importante, une preuve (pour Hedera).

3ases et policy.json

Les règles de dépense, versionnées.

- Une politique par organisation : `policy.json` pour l'ONG (2.1.0, limite 15 000 TND) et `policy.acme.json` pour ACME (1.1.0, limite 6 000 TND). Régions, catégories, plafonds, seuils de prix, poids du risque.
- `agents/guardian/engine.ts` : la fonction **evaluate**, 13 règles, sans IA. 20 tests unitaires la vérifient.
- La version effective est « 2.1.0+empreinte du fichier ».

Service de preuves

Transforme chaque décision en preuve chaînée et l'envoie à Hedera.

- `audit/proofService.ts` : JSON canonique → empreinte SHA-256 → maillon de chaîne (empreinte précédente) → message HCS.
- Une file d'attente par mission envoie les messages dans l'ordre, avec une nouvelle tentative si besoin.
- Vérification : relit le topic sur le Mirror Node et compare tout (événements modifiés, effacés, métadonnées changées).

Schémas partagés (packages/shared)

Le « contrat » commun entre le site et l'API.

- Écrits avec **Zod** : une mission, une offre, une décision… ont une forme précise, vérifiée à l'exécution.
- Le site et l'API utilisent les mêmes types : si l'un change, TypeScript signale l'erreur dans l'autre.

SQLite (node:sqlite)

La base de données locale, dans un fichier.

- Intégrée à Node.js (module `node:sqlite`), aucun serveur à installer.
- Tables : missions, offres, recommandations, vérifications, décisions, approbations, paiements, événements de preuve, messages.
- Des contraintes uniques empêchent deux paiements pour la même mission, la même facture ou la même clé d'idempotence.
- Code : `db/store.ts`. C'est cette base qu'un « initié » pourrait modifier : d'où Hedera.

API Claude (Anthropic)

L'IA, appelée par internet.

- SDK `@anthropic-ai/sdk`, modèle `claude-opus-5-5`, effort « low » pour aller vite, 45 s maximum, 1 nouvelle tentative.
- Réponse au format JSON imposé (schéma Zod).
- Utilisée par Lemhaf (classement) et par Lem3alem (texte libre seulement).

Nœuds Hedera (testnet)

Là où on écrit : preuves, paiements, coffre.

- SDK `@hashgraph/sdk`, connexion gRPC (port 50211).
- **TopicMessageSubmitTransaction** : envoyer une preuve au topic 0.0.11007004.
- **TransferTransaction** : payer en EJTND (jeton HTS) ou en HBAR le compte du fournisseur de démo.
- **ScheduleCreate / ScheduleSign / ScheduleDelete** : préparer, co-signer ou annuler un paiement depuis le coffre 2 sur 2.
- **TransactionReceiptQuery** : lire le reçu (SUCCESS).
- Clé des agents (compte 0.0.10569347) et clé de l'approbateur : dans .env, jamais affichées.
- Code : `hedera/sdkGateway.ts`. Mise en place : `npm run hedera:setup -- --vault --stablecoin`.

Vérificateur public (/verify)

Une page qui vérifie une mission sans faire confiance à notre serveur.

- Elle télécharge ce que le serveur affirme (`GET /api/public/missions/:id/bundle`) : chaque événement avec sa preuve complète.
- Elle lit les preuves **directement sur le Mirror Node** de Hedera, depuis ton navigateur.
- Elle recalcule chaque SHA-256 avec Web Crypto, vérifie la chaîne et compare avec Hedera. Elle vérifie aussi les paiements et le coffre.
- Résultat : VERIFIED ON HEDERA, ou TAMPERING DETECTED si quelqu'un (même nous) a modifié ou effacé l'historique.
- Code : `packages/shared/src/proof.ts` (verifyBundle) et `apps/web/components/PublicVerifier.tsx`.

Mirror Node (testnet)

Là où on lit l'historique, gratuitement.

- API web publique (REST) de Hedera : `testnet.mirrornode.hedera.com`.
- On y relit les messages du topic (pour Verify against Hedera) et les transactions (Mirror Node confirmed).
- HashScan, le site public, est construit sur ces mêmes données : n'importe qui peut vérifier sans nous.
- Code : `hedera/mirror.ts`.

CHAPITRE 5

Hedera, depuis zéro

Hedera est un registre public partagé, comme une blockchain. Des dizaines d'ordinateurs (les « nœuds »), gérés par de grandes organisations différentes, gardent chacun une copie des mêmes transactions. Pour qu'une transaction soit acceptée, ils doivent se mettre d'accord sur elle et sur son ordre : c'est le **consensus**. Une fois acceptée, elle est définitive : personne ne peut la modifier ni l'effacer, pas même celui qui l'a envoyée.

Hedera n'utilise pas une chaîne de blocs classique mais un algorithme appelé **hashgraph** : les nœuds se racontent en permanence ce qu'ils ont reçu (« gossip about gossip ») et en déduisent l'ordre des transactions sans vote explicite. Résultat : la décision est rapide (quelques secondes), les frais sont faibles et fixés en dollars, et l'ordre est horodaté pour tout le monde.

Il existe deux réseaux : le **mainnet** (vrai argent) et le **testnet** (argent de test gratuit). EL JOKER utilise uniquement le testnet.

LES NOTIONS, ET CE QUE NOUS EN UTILISONS

Compte (0.0.x) utilisé

Une identité sur Hedera, notée shard.realm.numéro. Notre compte opérateur est 0.0.10569347 ; le fournisseur de démo a aussi son compte.

Clé privée et signature utilisé

Chaque transaction est signée avec la clé privée du compte. Elle reste dans .env et ne doit jamais être partagée.

HBAR (ℏ) utilisé

La monnaie de Hedera. Elle paie les frais et sert à nos paiements de démo (montants réduits).

Transaction et frais utilisé

Une action envoyée au réseau (payer, envoyer un message…). Les frais sont fixés en dollars et très faibles.

Consensus et finalité utilisé

Les nœuds se mettent d'accord sur l'ordre en quelques secondes. Après, c'est définitif.

Reçu (receipt) utilisé

La réponse du réseau après consensus : SUCCESS ou une erreur. Kanawita ne marque PAID qu'avec SUCCESS.

Testnet / mainnet utilisé

Testnet = argent de test gratuit (portal.hedera.com). Mainnet = vrai argent. Nous : testnet uniquement.

HCS · Consensus Service utilisé

Le service qui reçoit des messages et leur donne un ordre et une heure officiels. C'est notre « notaire » : une preuve par décision.

Topic utilisé

Un fil de messages HCS. Le nôtre : 0.0.11007004. Seul notre compte peut y écrire (clé de soumission).

Numéro de séquence utilisé

Chaque message du topic reçoit un numéro (seq #) et un horodatage de consensus. Impossible d'en insérer un au milieu.

Mirror Node

utilisé

Des serveurs qui gardent tout l'historique et le donnent par une API web gratuite. On s'en sert pour vérifier.

HashScan

utilisé

Le site public pour explorer Hedera : transactions, comptes, topics. Le jury peut y voir nos preuves et notre paiement.

Transactions programmées

utilisé

Une transaction préparée à l'avance qui attend les signatures manquantes. C'est notre coffre : le paiement attend la signature humaine, puis Hedera l'exécute tout seul.

Clé à seuil (2 sur 2)

utilisé

Une clé faite de plusieurs clés, avec un nombre minimum de signatures. Le coffre exige la clé des agents ET celle de l'approbateur.

HTS · Token Service

utilisé

Créer ses propres jetons. Nous avons créé EJTND, un dinar de test (1 EJTND = 1 dinar, offre fixe, pas de clé admin), pour payer le montant complet. En vrai : un partenaire agréé, car le dinar ne circule pas librement hors de Tunisie.

Smart contracts

pas utilisé

Des programmes qui tournent sur Hedera (compatibles Ethereum). Pas utilisés : nos règles tournent dans notre API.

File Service

pas utilisé

Stocker des fichiers sur Hedera. Pas utilisé : nos documents restent hors chaîne, seule leur empreinte est publiée.

Ce qui est envoyé sur Hedera pour chaque décision (un message HCS)

Seulement des identifiants et des empreintes, moins de 1 Ko. Exemple réel de structure :

```
{ "p": "eljoker", "v": "1", "m": "msn_…", "e": "evt_…",
  "t": "GUARDIAN_APPROVED", "s": 3,
  "h":  "sha256 de la preuve complète (gardée chez nous)",
  "ph": "eventHash de l'événement précédent",
  "eh": "eventHash de cet événement",
  "pv": "2.1.0+…", "ts": "2026-10-12T…" }
```

> p = protocole, v = version, m = mission, e = événement, t = type, s = numéro dans la mission, h = empreinte des preuves, ph / eh = maillons de la chaîne, pv = version des règles, ts = heure. Aucun nom, aucun prix, aucun document.

CHAPITRE 6 · À MANIPULER

Empreintes, chaîne et falsification

C'est le cœur du lien avec Hedera. Une **empreinte SHA-256** (un « hash ») est un code de 64 caractères calculé à partir d'un texte. Le même texte donne toujours la même empreinte, mais changer une seule lettre donne une empreinte complètement différente. Et à partir de l'empreinte, impossible de retrouver le texte.

Essaie : calcule une empreinte

Écris ou modifie le texte (change juste un chiffre) :

Oasis Medical & Water Supply · total 12450 TND · facture n° 0001 (exemple)

Empreinte SHA-256 :

`8b90558830df1474404e05596b7ea2564a9427280432830c3e4606bf7ed71337`

Dans le projet, la preuve complète d'une décision (le JSON avec tous les détails) est d'abord mise en « JSON canonique » (toujours les clés dans le même ordre), puis on calcule son empreinte. C'est le champ `h` envoyé sur Hedera.

La chaîne d'une mission

Chaque événement contient l'empreinte du précédent. Si on modifie un événement, son empreinte change et le maillon suivant ne correspond plus. Une copie de chaque empreinte est sur Hedera.

1. MISSION_CREATED

Gabès · 500 personnes · budget 5000

`h 951f3ca483b2d185… · précédent 0000000000… · eh abfaa7fda4…`

2. AI_RECOMMENDATION

Oasis · 12450 TND

`h 0eb71681ecc16aad… · précédent abfaa7fda4… · eh c80228b220…`

3. GUARDIAN_APPROVED

approuvé · offre 12450 TND

`h 2c7c0399cdabd0cd… · précédent c80228b220… · eh d4b0049c6b…`

4. PAYMENT_CONFIRMED

12450 EJTND · SUCCESS

`h 3f3466f4bae3c56e… · précédent d4b0049c6b… · eh 5cc284fd15…`

> **Pourquoi c'est l'argument principal face au jury**
>
> Une vérification locale attrape l'attaque 1, mais pas les attaques 2 et 3 : un initié malin recalcule toutes les empreintes, ou efface simplement le dernier événement, et tout a l'air parfait. Seule la copie sur Hedera, que personne ne peut modifier, révèle la différence. C'est exactement ce que fait le bouton « Verify against Hedera » du site, en lisant le Mirror Node.

L'IA dans le projet

Quel modèle ?

Claude Opus 5.5 (identifiant `claude-opus-5-5`), d'Anthropic, appelé par internet avec la clé `ANTHROPIC_API_KEY` du fichier `.env`. Le code est dans `apps/api/src/ai/aiClient.ts`.

Ce que fait Lemhaf avec Claude

On lui envoie la mission et les offres (avec les totaux déjà calculés, pour qu'il ne refasse pas les calculs). Il renvoie un classement : rang, score, justification, inquiétudes, et les « faits » qu'il affirme (fournisseur, prix, délai).

Ce que fait Lem3alem avec Claude

Seulement si tu écris une demande libre : Claude transforme ton texte en mission structurée. Si tu envoies un scénario pré-rempli tel quel, la mission est déjà structurée et Claude n'est pas appelé (badge `DETERMINISTIC`).

Sortie structurée

On demande à Claude une réponse au format JSON, validée par un schéma **Zod**. Si la réponse ne respecte pas le schéma, elle est refusée. L'IA ne peut donc pas répondre « n'importe quoi » à la place des données attendues.

Si Claude ne répond pas

Après 45 secondes (une seule nouvelle tentative), on passe au classement par règles simples, affiché `FALLBACK · rule-based`. Le reste ne change pas : les contrôles et les règles sont les mêmes.

Pourquoi l'IA ne décide jamais

Une IA peut **halluciner** (inventer un fait) ou être **manipulée** (« prompt injection » : un texte caché qui lui donne des ordres). Donc Nasnes vérifie ses affirmations, 3ases décide sans IA, et on cache même à l'IA le statut « vérifié » des fournisseurs pour qu'elle ne puisse pas recopier la réponse.

> **Exemple réel (scénario D)**
>
> Avec un délai de 12 heures, Claude a choisi le courtier le moins cher et le plus rapide (Sahel Express, non vérifié). Il a même signalé lui-même un risque de fraude sur les prix. Mais ça n'a aucune importance : ce n'est pas lui qui décide. Nasnes rejette, 3ases bloque, aucun argent ne bouge.

CHAPITRE 8

Le business : à quoi ça sert dans le monde réel

Le problème

Les agents IA commencent à agir (commander, payer). Dès qu'ils dépensent de l'argent, trois risques : ils inventent, ils dépensent trop ou paient deux fois, et leurs traces peuvent être modifiées. Aujourd'hui on choisit entre autonomie et contrôle.

Pour qui

Toute organisation qui laisse un agent IA dépenser. D'abord les ONG et programmes de subventions (ils doivent justifier chaque dinar aux donateurs), puis les achats des entreprises. Les deux sont dans la démo.

La valeur

La rapidité d'un agent IA pour la routine, des règles de dépense impossibles à contourner, et une preuve publique que n'importe qui (donateur, auditeur) peut vérifier lui-même.

Qui paie (hypothèse)

Un abonnement par organisation plus un coût par preuve enregistrée. C'est une hypothèse : elle n'a pas encore été validée avec de vrais clients.

Ce qui est fictif aujourd'hui

Les fournisseurs et les deux organisations (marqués « demo »), le jeton EJTND, notre dinar de test (pas de valeur réelle), la clé de l'approbateur gardée par notre serveur, et le PIN opérateur (0619). Hedera, Claude et les paiements sur testnet, eux, sont réels.

La suite

Vrais fournisseurs et un partenaire agréé pour un vrai dinar numérique (avec l'accord de la Banque centrale) au lieu de notre jeton de test, puis des approbateurs qui signent depuis leur propre portefeuille, puis deux pilotes en Tunisie (une ONG, une entreprise), puis une bibliothèque de politiques par secteur.

CHAPITRE 9

Teste-toi avant le jury

Clique sur une réponse : l'explication s'affiche.

1. Au-dessus de la limite, qui peut faire partir l'argent du coffre ?

- Kanawita tout seul
- **Personne sans la signature de l'approbateur humain**
- Claude

Réponse : Personne sans la signature de l'approbateur humain. Le coffre a une clé 2 sur 2. Kanawita prépare la transaction programmée (1 signature), mais seule la signature de l'approbateur la fait exécuter par Hedera.

2. Que fait le vérificateur public ?

- Il demande à notre serveur si tout va bien
- **Ton navigateur lit Hedera lui-même et recalcule toutes les empreintes**
- Il envoie un e-mail au jury

Réponse : Ton navigateur lit Hedera lui-même et recalcule toutes les empreintes. Il ne fait pas confiance au serveur : il lit le Mirror Node depuis le navigateur et recalcule chaque SHA-256.

3. Qu'est-ce qui change entre l'ONG et ACME ?

- Les agents
- Le topic Hedera
- **Seulement le fichier de règles (politique)**

Réponse : Seulement le fichier de règles (politique). Mêmes agents, même topic, mêmes preuves. Seule la politique change : 15 000 TND pour l'ONG, 6 000 TND pour ACME, et d'autres catégories.

4. Qui décide si on paie ?

Claude (Lemhaf)

3ases, avec des règles fixes

Le site web

Réponse : 3ases, avec des règles fixes. C'est 3ases, du code sans IA. Claude ne fait que recommander, et Kanawita revérifie avant de payer.

5. Qu'est-ce qu'on envoie sur Hedera pour chaque décision ?

Toute la facture et les noms

Seulement des identifiants et des empreintes

Rien, Hedera sert juste à payer

Réponse : Seulement des identifiants et des empreintes. Moins de 1 Ko : identifiants, type d'événement, empreintes, version des règles, heure. Aucune donnée personnelle ni prix.

6. Un initié modifie le prix ET recalcule toutes les empreintes. Qui le voit ?

La vérification locale

Personne

La vérification contre Hedera

Réponse : La vérification contre Hedera. Localement la chaîne a l'air parfaite. Mais la copie sur Hedera ne peut pas être modifiée : la comparaison donne TAMPERING DETECTED.

7. Que veut dire le badge DETERMINISTIC ?

- **Résultat calculé par du code fixe, sans IA**
- Résultat de Claude
- Paiement confirmé

Réponse : Résultat calculé par du code fixe, sans IA. Mêmes données = même résultat, toujours. C'est le cas de Nasnes, 3ases et Kanawita.

8. Que vaut 1 EJTND, le jeton que Kanawita envoie ?

- 1 dollar
- 1 HBAR
- **1 dinar tunisien (jeton de test)**

Réponse : 1 dinar tunisien (jeton de test). EJTND est notre dinar de test sur Hedera : 12 450 TND d'offre = 12 450 EJTND payés. Il n'a pas de valeur réelle.

9. La mission est BLOCKED. Un opérateur peut-il l'approuver ?

- Oui, avec le PIN
- **Non, jamais**
- Oui, si Claude est d'accord

Réponse : Non, jamais. La validation humaine ne sert que pour les cas HUMAN REVIEW. Un blocage ne se contourne pas.

10. À quoi sert le Mirror Node ?

À envoyer les paiements

À relire l'historique Hedera gratuitement pour vérifier

À héberger le site

Réponse : À relire l'historique Hedera gratuitement pour vérifier. On écrit sur les nœuds Hedera, on relit sur le Mirror Node. HashScan utilise les mêmes données.

11. Que se passe-t-il si on rejoue exactement le même paiement ?

Il est payé deux fois

Il est refusé (HTTP 409) : toujours un seul paiement

Le site plante

Réponse : Il est refusé (HTTP 409) : toujours un seul paiement. Clé d'idempotence + contraintes uniques dans la base + Kanawita qui vérifie qu'aucun paiement n'existe déjà.